

Necessità $\rightarrow$ Rappresentare l'esecuzione di funzioni

$f: \text{Input} \rightarrow \text{Output}$

$\Rightarrow$ Vogliamo descrivere COME calcolare f
ovvero vogliamo rappresentare ALGORITMI

$\rightarrow$ Sequenza finita di passi discreti che descrivono come calcolare f

Strategie per il calcolo di f

$\Rightarrow$ Abbiamo bisogno di uno strumento che permette di manipolare dati descrivendo Algoritmi

PROGRAMMA

$\Rightarrow$ Linguaggi di programmazione (PL) : linguaggi formali che permettono di scrivere programmi

PROGRAMMI : Frodi ben formate di un PL, sono una rappresentazione FINITA di una sequenza potenzialmente INFINITA di effetti su una macchina

APPLICAZIONI

- Applicazioni scientifiche FORTRAN
- Applicazioni economiche COBOL
- Applicazioni AI LISP
- Applicazioni Sistemi C
- Software Web JAVA
- ⋮

LIVELLO

→ Basso Livello (Linguaggio macchina
↳ Assembly)

→ Alto Livello

↓
Paradigmi

→ Imperativo

→ Funzionale

→ ML, LISP

} Funzioni
Impure

$$f(x) = x^2 + x + 1$$

→ Logico

→ PROLOG

Procedurale ~> Imperativo / Strutturato
(procedura senza
Salti)

OOP ~> Java

Declarative → Logico / Funzionale
(base matematiche)

GENERAZIONI

- I: HW proprietario
- II: Assembly
- III: Alto livello : uso linguaggio naturale
- IV: SQL
- V: Orientato alle applicazioni : danno risposta a problemi specifici

CARATTERISTICHE

- sequenziali
- concorrenti
- parallelo
- distribuiti
- ∞
- scripting
- $\vdots$

LEGGI BILTÀ

WRITABILITY

AFFIDABILITÀ

→ quantità limitata di costrutti

→ poca moltiplicazione di elementi

→ poco overloading di operatori

Type checking

Gestione eccezioni

Aliasing (negativo)

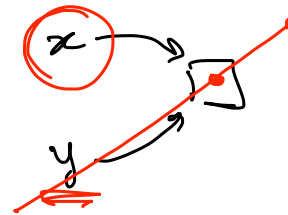
Ortogonalità

Tipi di dati

Sintassi

Astrazione (∞)

Espressività



IMPLEMENTARE UN PL

↳ Architettura delle macchine

↳ Metodologie di progettazione di programmi

↳ Dati e programmi in memoria (separata dalla CPU)

Istruzioni e dati che viaggiano tra memoria e CPU

```
program_counter
repeat forever
    fetch instruction nella memoria usando program_counter
    incrementa program_counter
    decode instruction
    execute instruction
end repeat
```

Implementare PL significa costruire la macchina astratta
per il PL

Macchine astratte programmi in alto livello

Macchine fisiche programmi in macchina

PL $\leadsto$ Linguaggio L con i suoi costrutti sintattici

↓

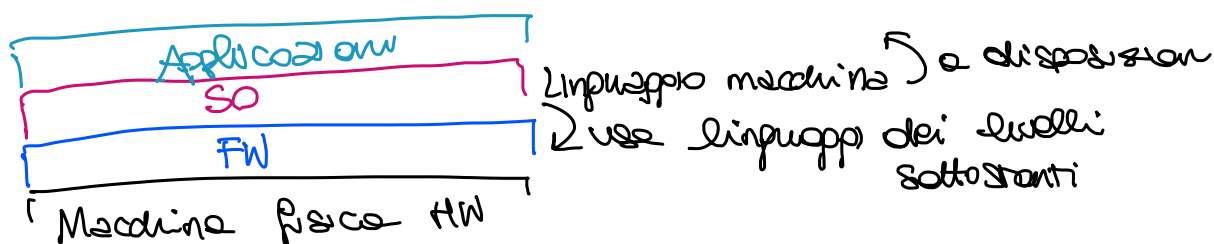
Programmi (frase ben formata nel PL)

Per eseguire costruiamo la macchina astratta per L

MACCHINA ASTRATTA Dato L linguaggio di programmazione
 M_L macchina astratta di L è un
 insieme di strutture dati e
 algoritmi che permettono di eseguire
 programmi scritti in L

LINGUAGGIO MACCHINA Dato M macchina astratta se
 linguaggio L_M è chiamato linguaggio
 macchina di M , ed è l'insieme di
 tutte le stringhe eseguibili (riconoscibili)
 da M .

Implementare un PL $\rightarrow$ Realizzare M_L



Implementazione di L

Traduzione
 implicita
 (Interprete)



Realizziamo M_L

Simulando ogni istruzione di L
 nel linguaggio di M_{L0} (macchine
 sottostanti)

Traduzione
 esplicita
 (Compilatore)

Realizziamo M_L
 traducendo internamente
 il programma scritto in L
 in un programma scritto in L_0

Notazione

Prog^L = insieme dei programmi scritti in L

D = insieme dei dati

$P^L \in \text{Prog}^L$ programma scritto in L

$$P^L : D \rightarrow D$$

input $\in D$ $P^L(\text{input}) = \text{output} \in D$

esecuzione di P^L

sul dato input restituisce il dato output

INTERPRETE

Un interprete, per il linguaggio L , scritto in L_0 (interprete viene scritto in un linguaggio implementato da M_0 sottostante) e' un programma

$$I^{L_0 L} : \text{Prog}^L \times D \rightarrow D$$

$$\forall \text{input} \in D. I^{L_0 L}(P^L, \text{input}) = \text{output}$$

$$\forall P^L \in \text{Prog}^L$$

Macchina
Universale

